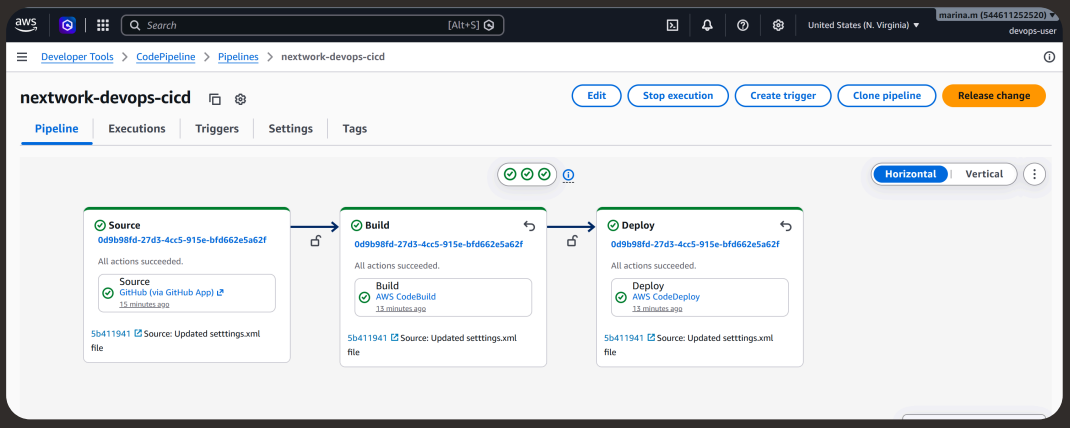
BUILD A CI/CD PIPELINE WITH AWS

By Maryna Maksimuk | October 2026



Introducing Today's Project!

Today, I'm building an end-to-end CI/CD pipeline using AWS CodePipeline by connecting GitHub for source control, CodeBuild for automated builds, and CodeDeploy for application deployment. I'll then run the pipeline to verify that code changes can move automatically from the repository through the build process to deployment on EC2.

Key tools and concepts

I learned how to build an end-to-end CI/CD pipeline using AWS CodePipeline, GitHub, CodeBuild, CodeDeploy, and Amazon S3. I also learned about IAM service roles and permissions, webhook triggers, pipeline execution modes, automated deployments, monitoring pipeline stages, and rolling back to a previous application version.

Project reflection

It took me approximately 3 hours to complete this project.
The most rewarding part was seeing the entire CI/CD pipeline work end to end. It was also a great refresher on pipeline concepts and strengthened my knowledge of AWS, automated deployments, troubleshooting, and rollbacks.

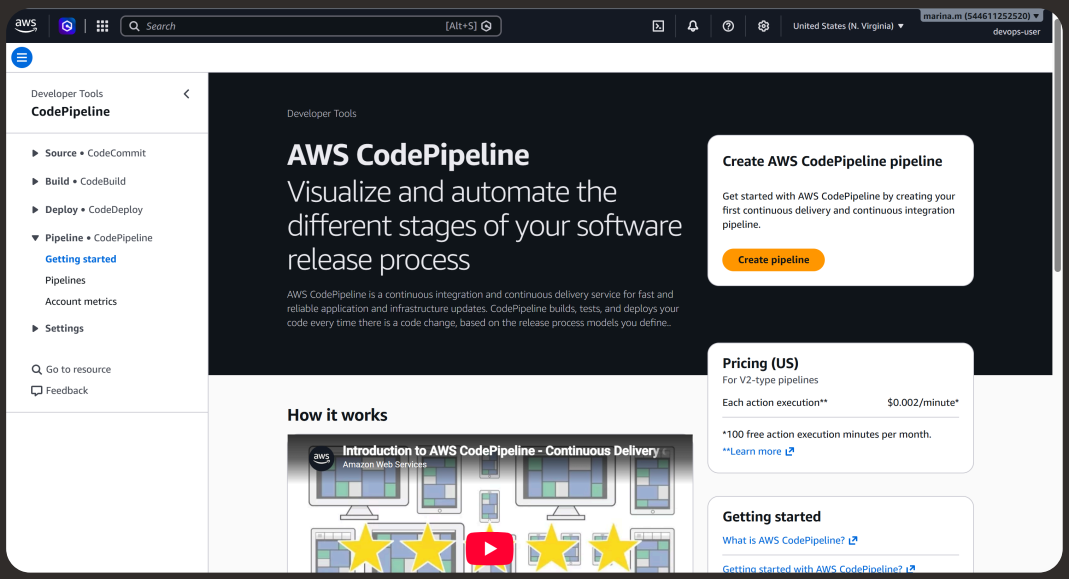
I chose to do this project today because I wanted to refresh my CI/CD pipeline knowledge and get more hands-on practice with AWS deployment automation.

Starting a CI/CD Pipeline

AWS CodePipeline is a managed CI/CD service that automates software delivery by connecting source control, build, testing, and deployment stages into one workflow.

AWS CodePipeline has three execution modes: SUPERSEDED, where newer executions can replace older ones waiting between stages; QUEUED, where executions run one at a time in order; and PARALLEL, where multiple executions run independently at the same time.

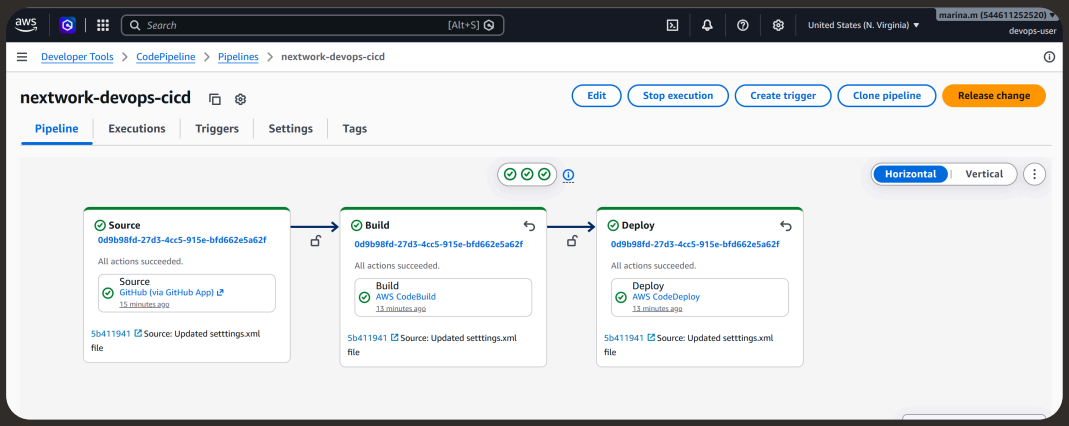
AWS CodePipeline has a service role so it can securely access other AWS services needed to run the pipeline, such as S3, CodeBuild, and CodeDeploy. The role grants CodePipeline the permissions it needs without requiring you to store AWS credentials directly in the pipeline configuration.



CI/CD Stages

My AWS CodePipeline, named nextwork-devops-cicd, has three stages:
1. Source: Connects to my GitHub repository MarynaMaksimuk/nextwork-web-project and monitors the main branch for code changes, automatically detecting changes and producing a ZIP artifact.
2. Build: Uses the AWS CodeBuild project nextwork-cicd-codebuild to build and validate the application, with automatic retry enabled if the stage fails.
3. Deploy: Uses AWS CodeDeploy application nextwork-cicd-codedeploy and deployment group nextwork-cicd-deploymentgroup to deploy the application, with automatic rollback enabled if deployment fails.
The pipeline uses V2 with SUPERSEDED execution mode, an automatically created S3 bucket for artifacts, and an AWS service role to access the required services.

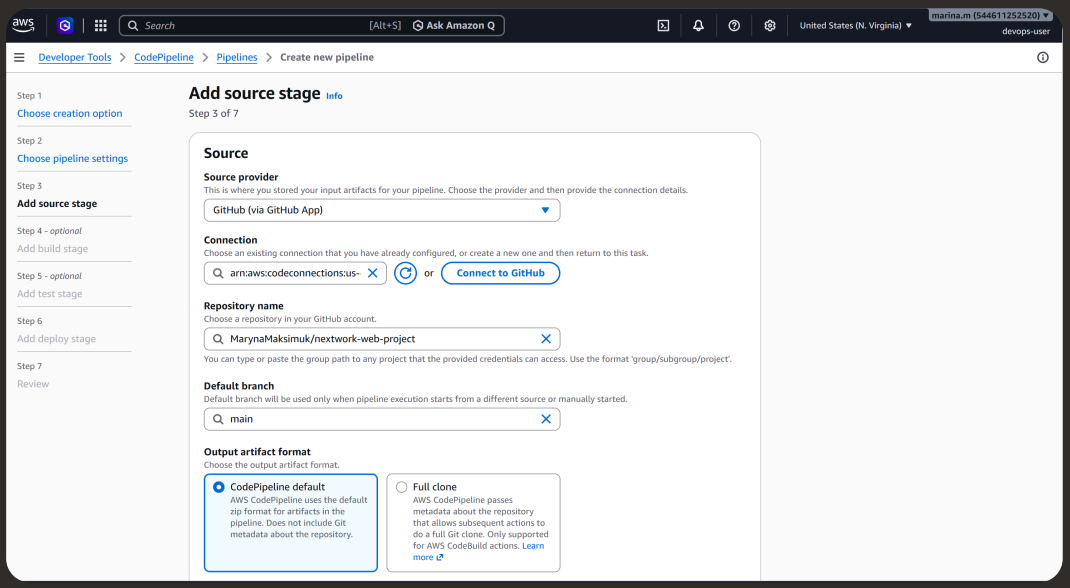
My pipeline shows the status of each stage (Source, Build, and Deploy), whether it succeeded or failed, and details about execution progress, errors, and logs. This helps me identify where a problem occurred and troubleshoot the CI/CD process.



Source Stage

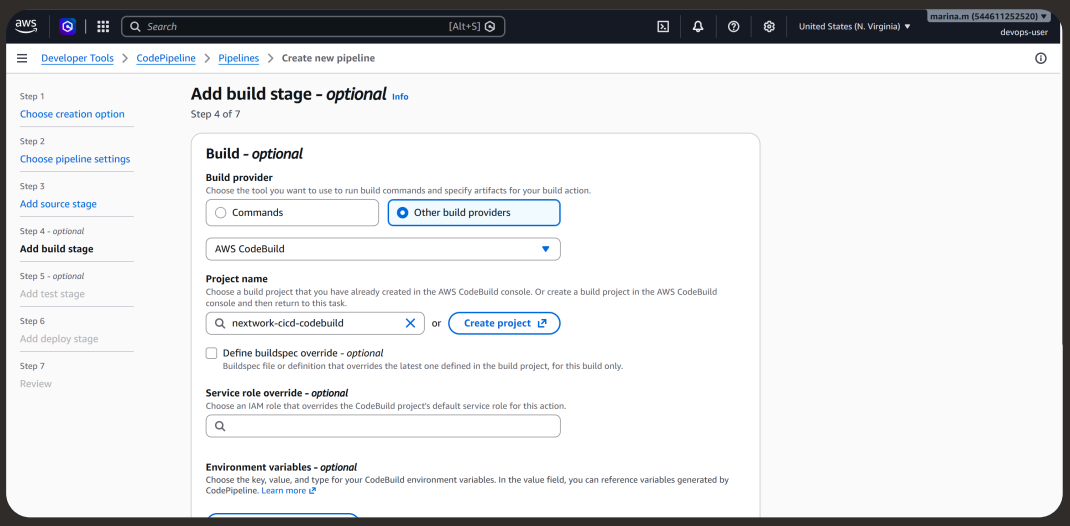
The default branch (main) tells AWS CodePipeline which branch in my GitHub repository to monitor for code changes and use as the source for the pipeline.

Webhook events automatically trigger the CI/CD pipeline when changes are pushed to GitHub, so the code can be built, tested, and deployed without manually starting the pipeline.



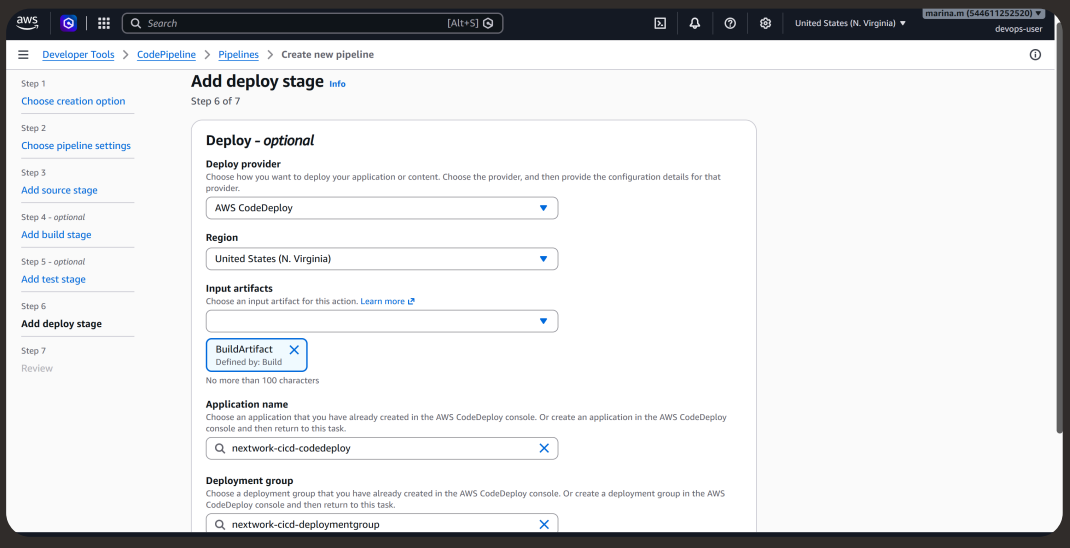
Build Stage

SourceArtifact contains the source code retrieved from GitHub, which CodeBuild needs as input to build and test the application.



Deploy Stage

I configured the Deploy stage to use AWS CodeDeploy, connecting it to my CodeDeploy application and deployment group so the built application can be deployed to my EC2 instance.

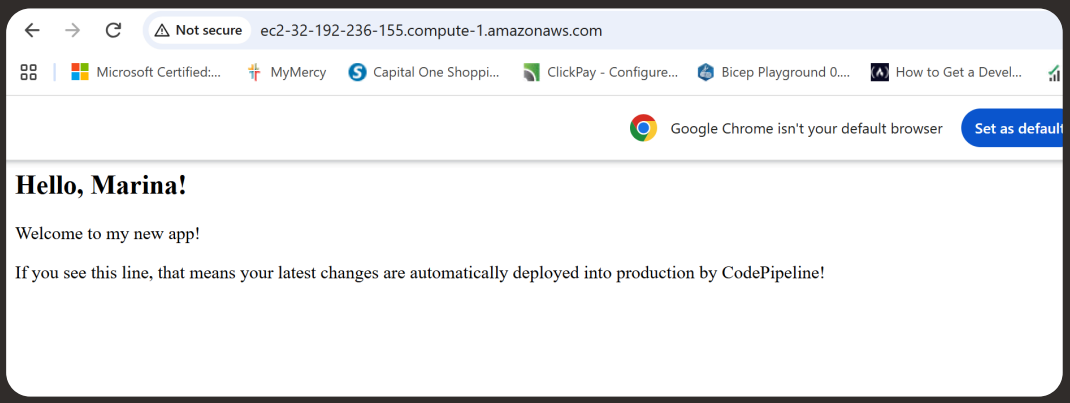


Success!

I updated the index.jsp file by adding a paragraph confirming that the latest code changes are automatically deployed to production through AWS CodePipeline. This change helps me verify that pushing code to GitHub triggers the pipeline and deploys the updated application successfully.

After I pushed the code change to GitHub, AWS CodePipeline automatically detected the update and triggered the Source, Build, and Deploy stages. Once the pipeline completed successfully, the updated application was deployed to my EC2 instance, allowing me to verify the change in the browser.

I confirmed the CI/CD pipeline was working by checking that all three stages—Source, Build, and Deploy—completed successfully, then opening the application in my browser to verify that the new message I added to index.jsp appeared on the page.



Testing the Pipeline

In a project extension, I initiated a rollback on the Deploy stage to revert the application to its previous working version

The Source and Build stages are unaffected because the manual rollback was initiated only in the Deploy stage. The source code remains in GitHub, and the build artifacts remain available; the rollback affects the deployed application version, not the earlier stages.

After the rollback, I observed that my web application returned to its previous working version, and the latest code changes were no longer visible. This confirmed that AWS CodeDeploy successfully restored the previous application revision.

